Lab 7

1. A 5-meter ladder is initially standing upright against a wall. Its bottom end is at (0,0), and its top end is at (0,5).

a. Now, the ladder undergoes the following transformations:. It is tilted 30◦ counterclockwise around its bottom end (0,0).

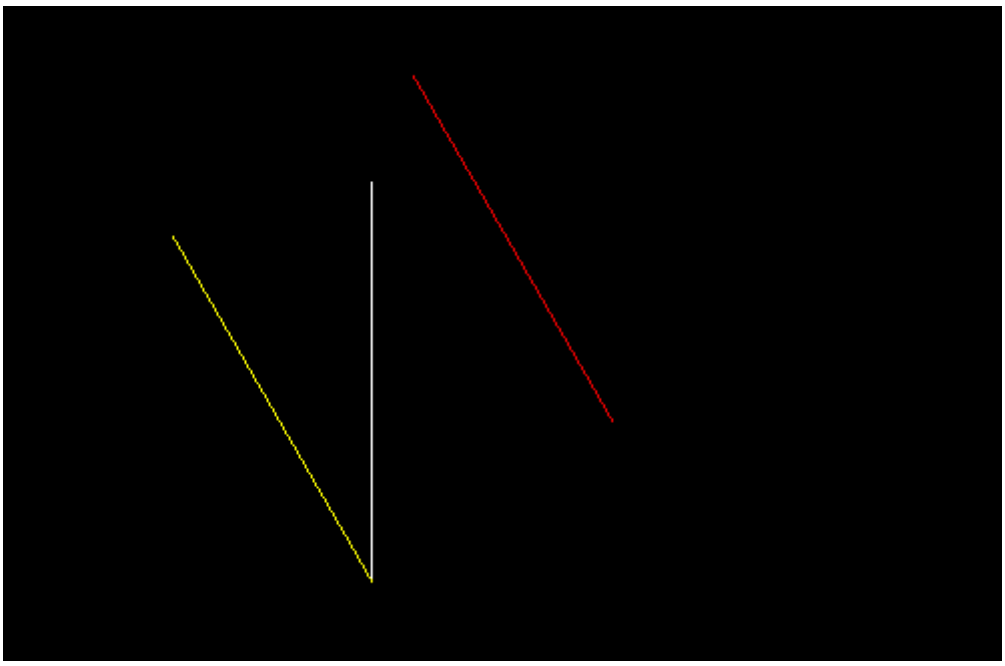
b. Then, the entire ladder is moved (translated) so that its bottom end is now at (3,2).

Apply necessary transformations and obtain the desired final figure.

```
#include <GL/glut.h>
#include <cmath>

float angle = 30.0;

void drawLine(float x1, float y1, float x2, float y2) {
   glBegin(GL_LINES);
   glVertex2f(x1, y1);
   glVertex2f(x2, y2);
   glEnd();
}

void display() {
   glClear(GL_COLOR_BUFFER_BIT);

   float theta = angle * M_PI / 180.0;

   glColor3f(1.0, 1.0, 1.0);
   drawLine(0.0, 0.0, 0.0, 5.0);

   float x = 0.0, y = 5.0;
```

```
    float xr = x * cos(theta) - y * sin(theta);
    float yr = x * sin(theta) + y * cos(theta);

    glColor3f(1.0, 1.0, 0.0);
    drawLine(0.0, 0.0, xr, yr);



    float tx = 3.0, ty = 2.0;

    float x1 = tx;
    float y1 = ty;

    float x2 = xr + tx;
    float y2 = yr + ty;

    glColor3f(1.0, 0.0, 0.0);
    drawLine(x1, y1, x2, y2);

    glFlush();
}

void init() {
    glClearColor(0.0, 0.0, 0.0, 1.0);

    glMatrixMode(GL_PROJECTION);
    gluOrtho2D(-5, 10, -5, 10);
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(600, 600);
    glutCreateWindow("Rotation and Translation of Ladder");
```

```
    init();
    glutDisplayFunc(display);
    glutMainLoop();

    return 0;
}
```



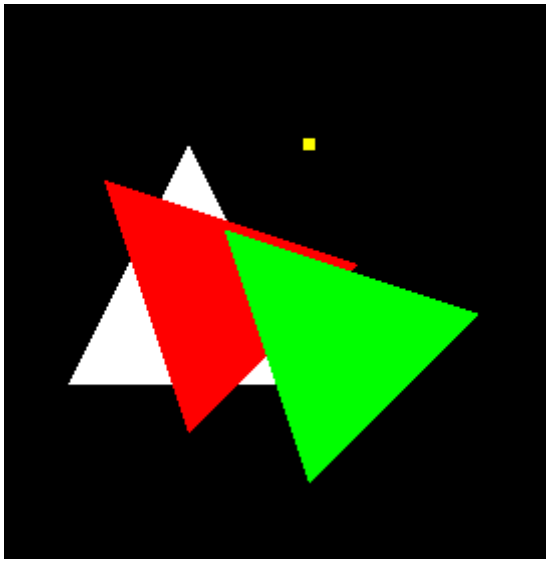
**2.** Create and rotate a triangle about the origin and a fixed point.

```
#include <GL/glut.h>
#include <cmath>

float angle = 45.0;


float tri[3][2] = {
    {0.0, 1.0},
    {-1.0, -1.0},
    {1.0, -1.0}
};
```

```
float px = 1.0, py = 1.0;


float toRad(float deg) {
    return deg * 3.14159 / 180.0;
}


void drawOriginal() {
    glColor3f(1, 1, 1);
    glBegin(GL_TRIANGLES);
    for (int i = 0; i < 3; i++)
        glVertex2f(tri[i][0], tri[i][1]);
    glEnd();
}

void drawRotateOrigin() {
    float rad = toRad(angle);

    glColor3f(1, 0, 0);
    glBegin(GL_TRIANGLES);
    for (int i = 0; i < 3; i++) {
        float x = tri[i][0];
        float y = tri[i][1];

        float xr = x * cos(rad) - y * sin(rad);
        float yr = x * sin(rad) + y * cos(rad);

        glVertex2f(xr, yr);
    }
```

```
    glEnd();
}

void drawRotateFixed() {
    float rad = toRad(angle);

    glColor3f(0, 1, 0);
    glBegin(GL_TRIANGLES);
    for (int i = 0; i < 3; i++) {
        float x = tri[i][0];
        float y = tri[i][1];

        float xr = px + (x - px) * cos(rad) - (y - py) * sin(rad);
        float yr = py + (x - px) * sin(rad) + (y - py) * cos(rad);

        glVertex2f(xr, yr);
    }
    glEnd();
}

void drawPivot() {
    glPointSize(6);
    glColor3f(1, 1, 0);
    glBegin(GL_POINTS);
    glVertex2f(px, py);
    glEnd();
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);

    drawOriginal();
```

```
    drawRotateOrigin();
    drawRotateFixed();
    drawPivot();

    glFlush();
}

void init() {
    glClearColor(0, 0, 0, 1);
    gluOrtho2D(-5, 5, -5, 5);
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(600, 600);
    glutCreateWindow("Manual Rotation (No Built-in)");

    init();
    glutDisplayFunc(display);

    glutMainLoop();
    return 0;
}
```

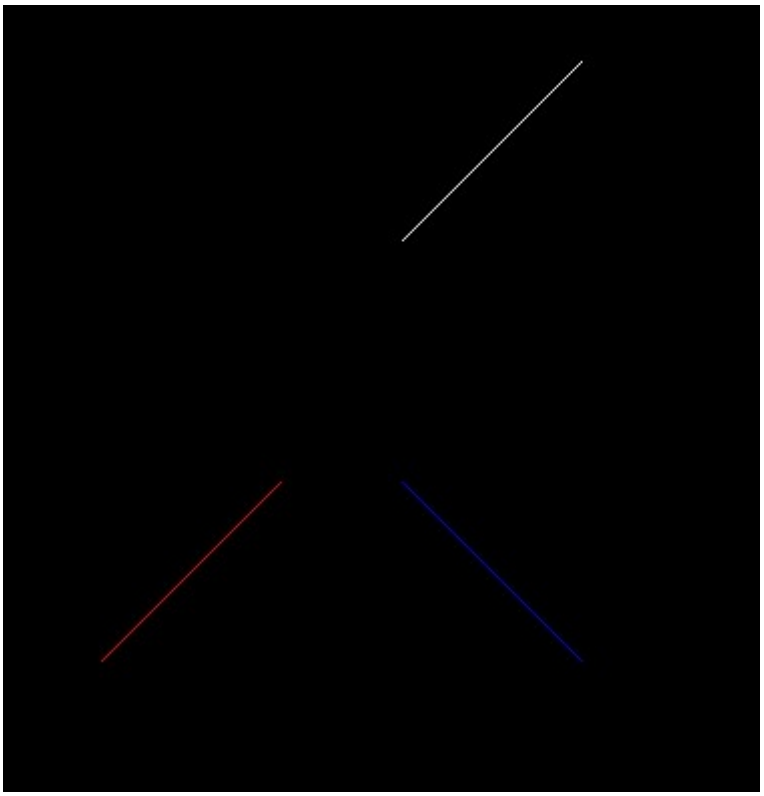


**3.** Reflect a line first about the x-axis and then about the y-axis.

```
#include <GL/glut.h>

// Original line
float xA = 1, yA = 2;
float xB = 4, yB = 5;

// After X-axis reflection
float xA_x, yA_x, xB_x, yB_x;

// After Y-axis reflection (final)
float xA_xy, yA_xy, xB_xy, yB_xy;

void computeReflection() {
    // Step 1: Reflect about X-axis
    xA_x = xA;
    yA_x = -yA;

    xB_x = xB;
    yB_x = -yB;

    // Step 2: Reflect about Y-axis
    xA_xy = -xA_x;
    yA_xy = yA_x;

    xB_xy = -xB_x;
    yB_xy = yB_x;
}

void drawLine(float x1, float y1, float x2, float y2) {
    glBegin(GL_LINES);
    glVertex2f(x1, y1);
```

```
    glVertex2f(x2, y2);
    glEnd();
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);

    // 1 Original line (White)
    glColor3f(1, 1, 1);
    drawLine(xA, yA, xB, yB);

    // 2 Reflection about X-axis (Blue)
    glColor3f(0, 0, 1);
    drawLine(xA_x, yA_x, xB_x, yB_x);

    // 3 Reflection about X then Y (Red)
    glColor3f(1, 0, 0);
    drawLine(xA_xy, yA_xy, xB_xy, yB_xy);

    glFlush();
}

void init() {
    glClearColor(0, 0, 0, 1);
    gluOrtho2D(-10, 10, -10, 10);

    computeReflection();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(600, 600);
```

```
    glutCreateWindow("Reflection: X-axis then Y-axis");

    init();
    glutDisplayFunc(display);

    glutMainLoop();
    return 0;
}
```



**4.** You are given the following geometric shapes in a local coordinate system:

a. A rectangle (representing the house) with bottom-left corner at (0,0) and top-right corner at (6,9).

b. A triangle (representing the roof) with vertices at: A=(0,0), B=(2,0), C=(1,1) i. You need to:

c. Translate the rectangle so that its left edge is positioned at x=5 on the x-axis.

d. Place the triangle on top of the rectangle, ensuring that:

1. The base of the triangle aligns with the top edge of the rectangle.

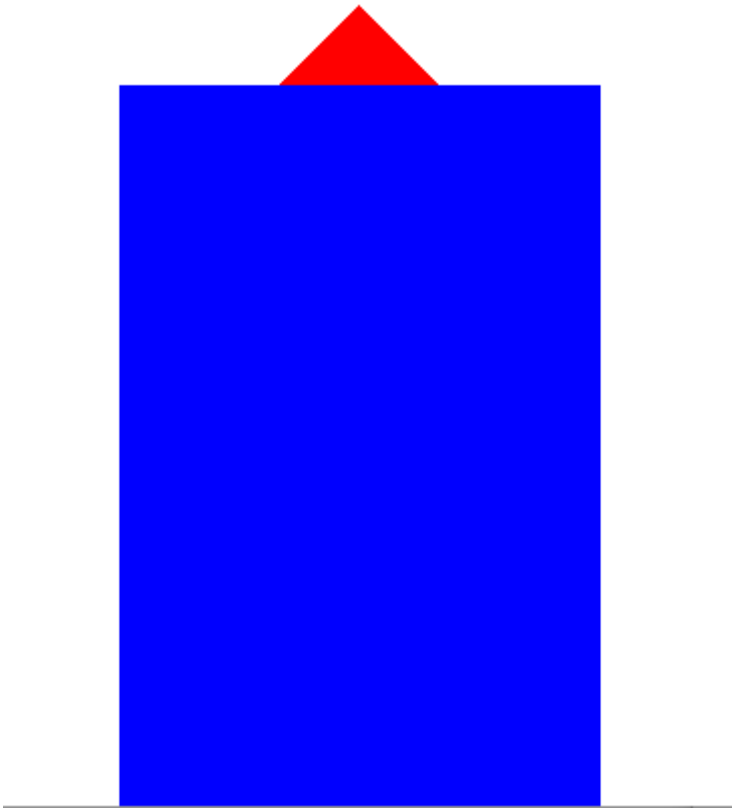
ii. The triangle remains centered over the rectangle.

```
' Apply the transformations and show the final figure
#include <GL/glut.h>
```

```
float rect[4][2] = {
    {0, 0}, {6, 0}, {6, 9}, {0, 9}
};

float tri[3][2] = {
    {0, 0}, {2, 0}, {1, 1}
};

float rectT[4][2];
float triT[3][2];

void computeTransform() {
    for (int i = 0; i < 4; i++) {
        rectT[i][0] = rect[i][0] + 5;
        rectT[i][1] = rect[i][1];
    }

    float rectCenterX = (rectT[0][0] + rectT[2][0]) / 2;

    float triCenterX = (tri[0][0] + tri[1][0]) / 2;

    float dx = rectCenterX - triCenterX;
    float dy = rectT[2][1] - tri[0][1];

    for (int i = 0; i < 3; i++) {
        triT[i][0] = tri[i][0] + dx;
        triT[i][1] = tri[i][1] + dy;
    }
}

void drawPolygon(float vertices[][2], int n, float r, float g, float b) {
    glColor3f(r, g, b);
```

```
    glBegin(GL_POLYGON);
    for (int i = 0; i < n; i++)
        glVertex2f(vertices[i][0], vertices[i][1]);
    glEnd();
}

void display() {
    glClear(GL_COLOR_BUFFER_BIT);

    drawPolygon(rectT, 4, 0, 0, 1);

    drawPolygon(triT, 3, 1, 0, 0);

    glFlush();
}

void init() {
    glClearColor(1, 1, 1, 1);
    gluOrtho2D(0, 15, 0, 15);
    computeTransform();
}

int main(int argc, char** argv) {
    glutInit(&argc, argv);
    glutInitWindowSize(600, 600);
    glutCreateWindow("House with Roof");

    init();
    glutDisplayFunc(display);

    glutMainLoop();
    return 0;
```

}



**5.** Write a program to implement **Boundary Fill Algorithm using recursion (8-connected).**

**6.** Write a program for **Flood Fill Algorithm**

**7.** Write a program to:

- Draw a polygon
- Fill it using Boundary Fill
- Allow user to click seed point using mouse